

CXR Kubernetes Deployment Manual

Bootcamp workflow: Terraform, kind, Helm, and Argo CD

`cxr-ops-lab`

2026-05-29

Abstract

This manual documents the end-to-end Kubernetes deployment path used for the CXR bootcamp: local **kind** cluster **cxr-lab**, Docker image **cxr-ui:local**, Helm chart **helm/cxr-ui**, optional Terraform provisioning, and Argo CD GitOps from **GitHub** (UdonsiKalu/**cxr-ops-lab**). It explains rationale, folder layout, commands, ports, CI vs. CD, and how to adapt the pattern to another environment.

Contents

1	Executive summary	2
1.1	What you built	2
1.2	One-line flow	2
1.3	URLs (lab)	2
2	Rationale: why each layer exists	3
2.1	Design principles	3
2.2	CI vs. CD (critical distinction)	3
3	Repository and folder map	6
3.1	cxr-ops-lab layout (canonical)	6
3.2	Related repos (not merged)	6
4	Architecture diagrams	7
4.1	End-to-end deployment flow	7
4.2	In-cluster object graph (Argo tree)	7
4.3	Layered deployment model (host + cluster + Git)	8
4.4	Dependencies (M4.8): out of cluster	9
5	Step-by-step: run from zero	14
5.1	Prerequisites	14
5.2	Option A — Full end-to-end (recommended)	14
5.3	Option B — Manual layers	14
5.4	Argo CD login	14
5.5	Verify	14
6	Helm and GitOps workflow	16
6.1	Helm role	16
6.2	Day-to-day change loop	16
6.3	What is not “set and forget”	16
7	Script reference	17
8	Adapting to another deployment	18
8.1	Checklist	18
8.2	Syllabus mapping	18
9	Troubleshooting	19
10	Evidence and further reading	20

Chapter 1

Executive summary

1.1 What you built

- A **local Kubernetes lab** (`kind / cxr-lab`) running the rehearsal CXR UI.
- **Helm (SW.4)** packages the app as chart `cxr-ui`.
- **Terraform (SW.5)** can reproducibly ensure the kind cluster exists.
- **Argo CD (SW.8)** is **continuous delivery**: Git on GitHub is the source of truth; Argo syncs the Helm chart into the cluster.
- **CI** (`build/test/Trivy`) lives in `cxr-ui-rehearsal` — it does **not** deploy to this kind cluster.

1.2 One-line flow

Build image → Load into kind → Helm deploy → Argo keeps Git = cluster → Browser via port-forward
--

1.3 URLs (lab)

URL	Meaning
<code>http://localhost:8251</code>	Daily dev (full analyze)
<code>http://localhost:3000</code>	Docker Compose lab (SW.2)
<code>http://localhost:8081</code>	K8 UI (port-forward to pod :3000)
<code>https://localhost:8083</code>	Argo CD UI

Chapter 2

Rationale: why each layer exists

2.1 Design principles

1. **Do not touch production** cxrlabs-dev/platform/infra.
2. **Same app image path** as SW.1 Docker: rehearsal cxr-ui built into cxr-ui:local.
3. **Learn K8 mechanics** (namespace, deployment, service) before GitOps polish.
4. **Helm** parameterizes the chart (image tag, replicas, env) for SW.4.
5. **Terraform** documents reproducible cluster creation (SW.5 ADR: local kind only for boot-camp).
6. **Argo CD** implements SW.8 GitOps: push chart changes $\Rightarrow$ cluster reconciles.

2.2 CI vs. CD (critical distinction)

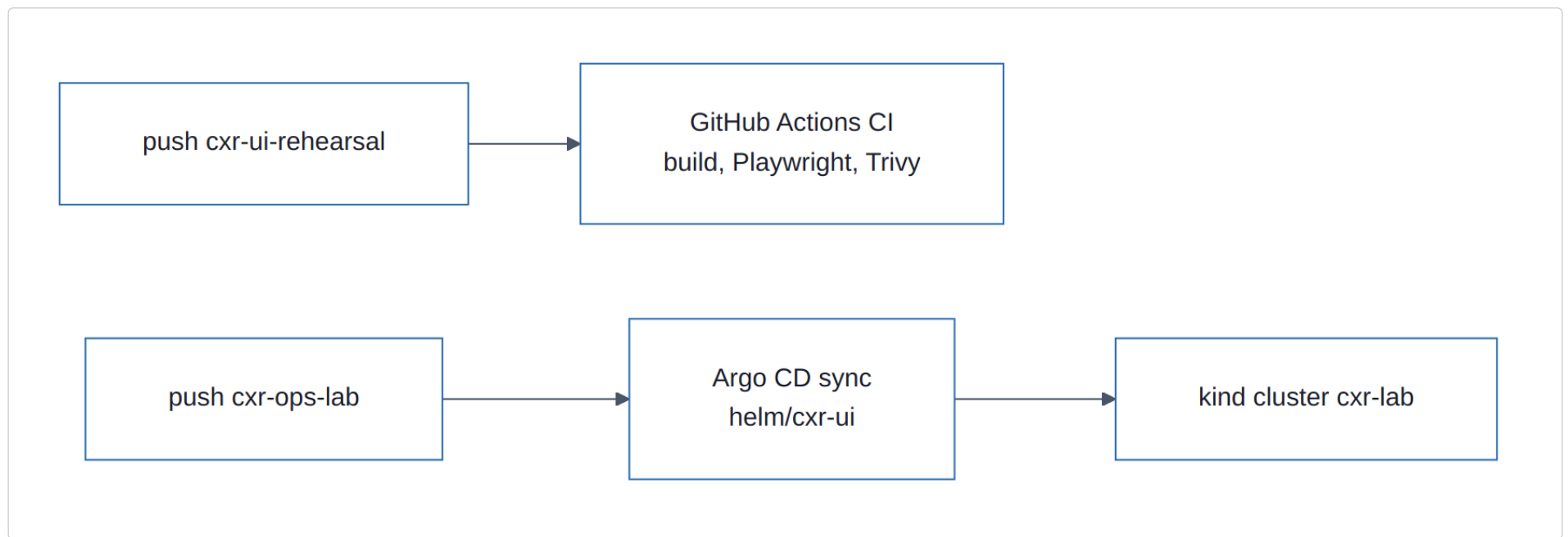


Figure 2.1: *CI (rehearsal repo) vs. CD (Argo sync from cxr-ops-lab Git)*

Push-based CD (GitHub Actions runs `helm upgrade`) is a valid alternative but **not wired** for this home lab because the cluster is local, the image is `cxr-ui:local` loaded via `kind load`, and the syllabus target for CD is Argo.

Chapter 3

Repository and folder map

3.1 cxr-ops-lab layout (canonical)

```
cxr-ops-lab/  
  Dockerfile                # SW.1 image (cxr-ui:local)  
  Dockerfile.compose        # SW.2 (not used in K8 pod by default)  
  helm/cxr-ui/              # SW.4 Helm chart (Argo source path)  
    Chart.yaml  
    values.yaml  
    templates/  
  k8s/                      # SW.3 raw manifests (study / --raw)  
    namespace.yaml  
    deployment.yaml  
    service.yaml  
    argocd/  
      application-cxr-ui.yaml  
  terraform/                # SW.5 kind provisioner  
  scripts/  
    00-install-tools.sh     # kind, kubectl, helm, terraform, argocd  
    01-kind-cluster.sh  
    02-build-and-load.sh  
    03-k8-up.sh             # one-shot cluster+image+helm  
    05-helm-install.sh  
    12-k8-ensure.sh  
    13-argo-install.sh  
    14-stack-test.sh  
    15-e2e-deploy.sh        # full stack  
  evidence/                 # SW.3, SW.4, SW.8 proof  
  docs/                     # this manual  
  bin/                      # downloaded CLIs (gitignored)
```

3.2 Related repos (not merged)

- **App source:** `cxr-ui-prune-rehearsal/cxr-ui` (build context for Dockerfile).
- **CI:** <https://github.com/UdonsiKalu/cxr-ui-rehearsal>.
- **Ops + Helm + Argo:** <https://github.com/UdonsiKalu/cxr-ops-lab> (private).
- **Production (untouched):** `cxrlabs-dev/claim_analysis_tools/platform/infra`.

Chapter 4

Architecture diagrams

Diagram sources live in `docs/diagrams/*.mmd` (Mermaid). Render with `./scripts/render-k8-manual-diagram` before building the PDF.

4.1 End-to-end deployment flow

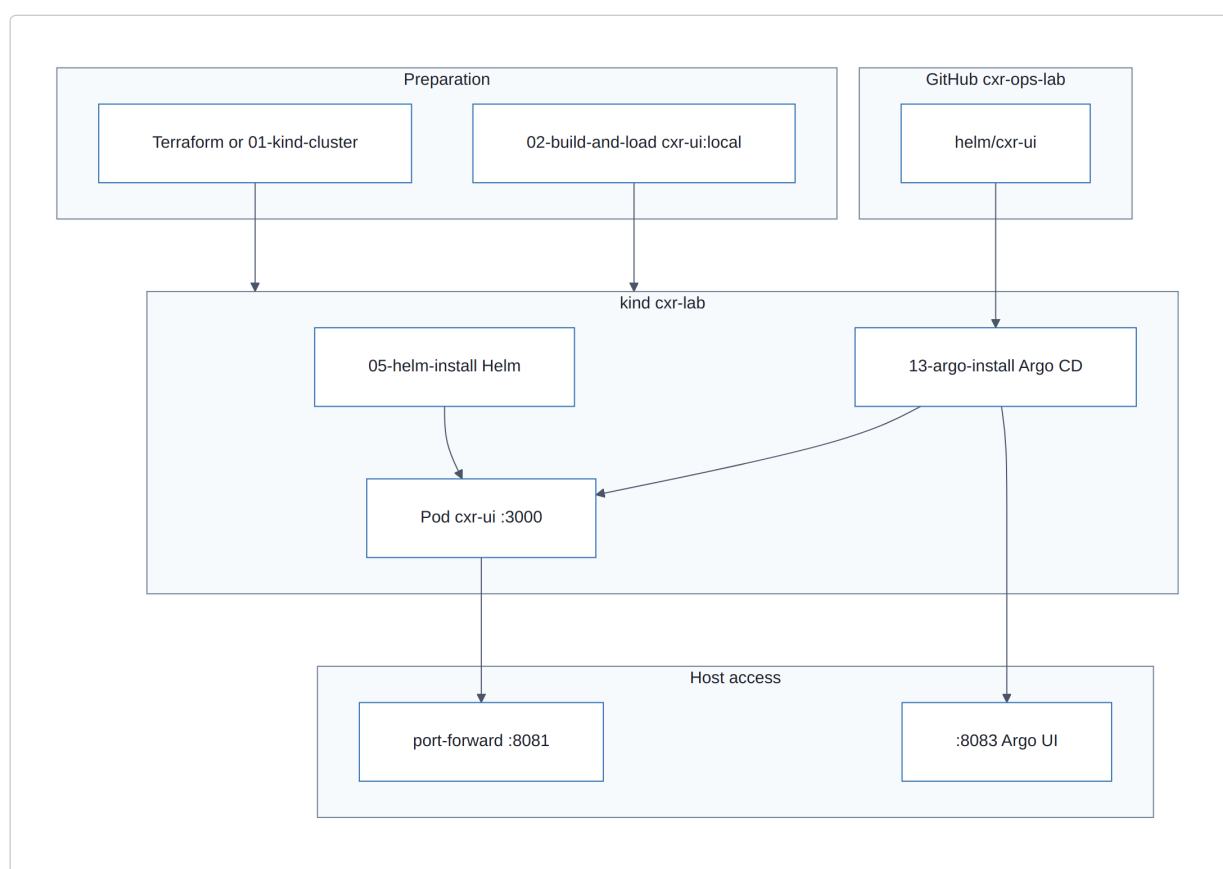


Figure 4.1: Full stack: Terraform/kind, image load, Helm, Argo CD, GitHub chart path, and host port-forwards

4.2 In-cluster object graph (Argo tree)

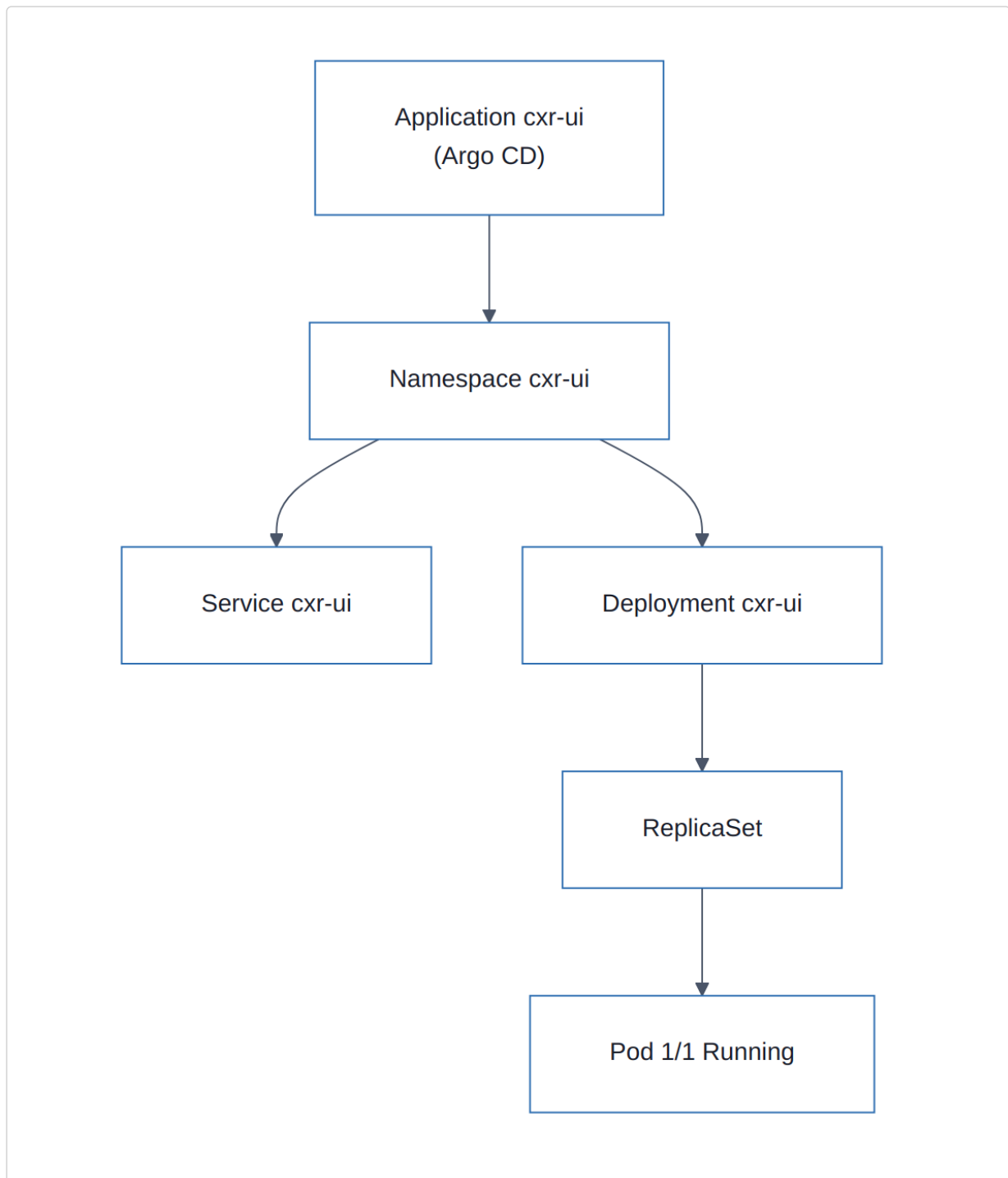


Figure 4.2: *Argo Application `cxx-ui` and Kubernetes resources (Synced/Healthy tree)*

4.3 Layered deployment model (host + cluster + Git)

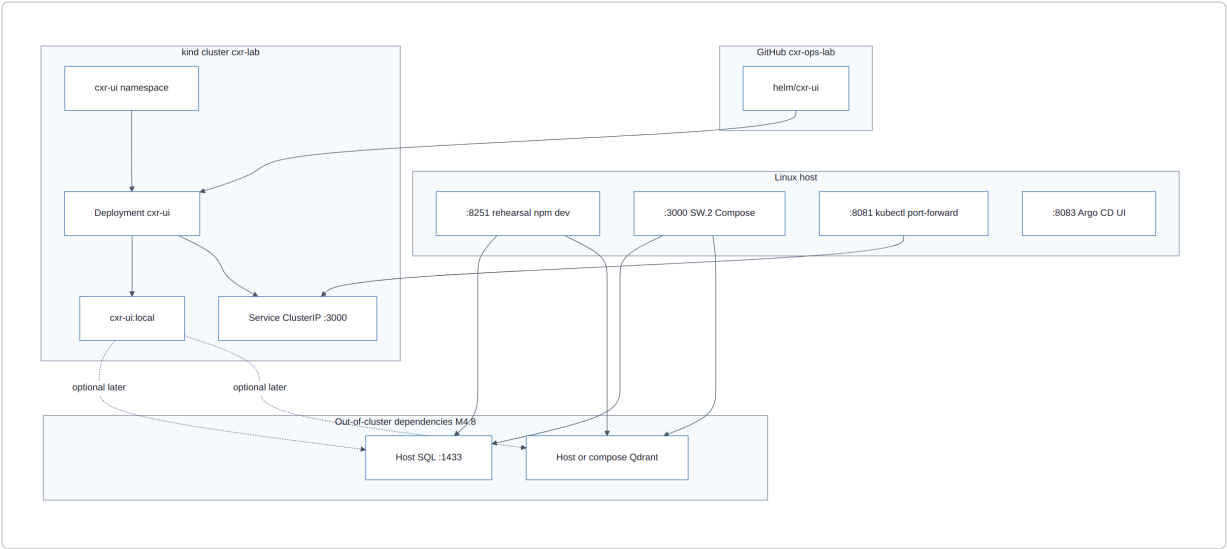
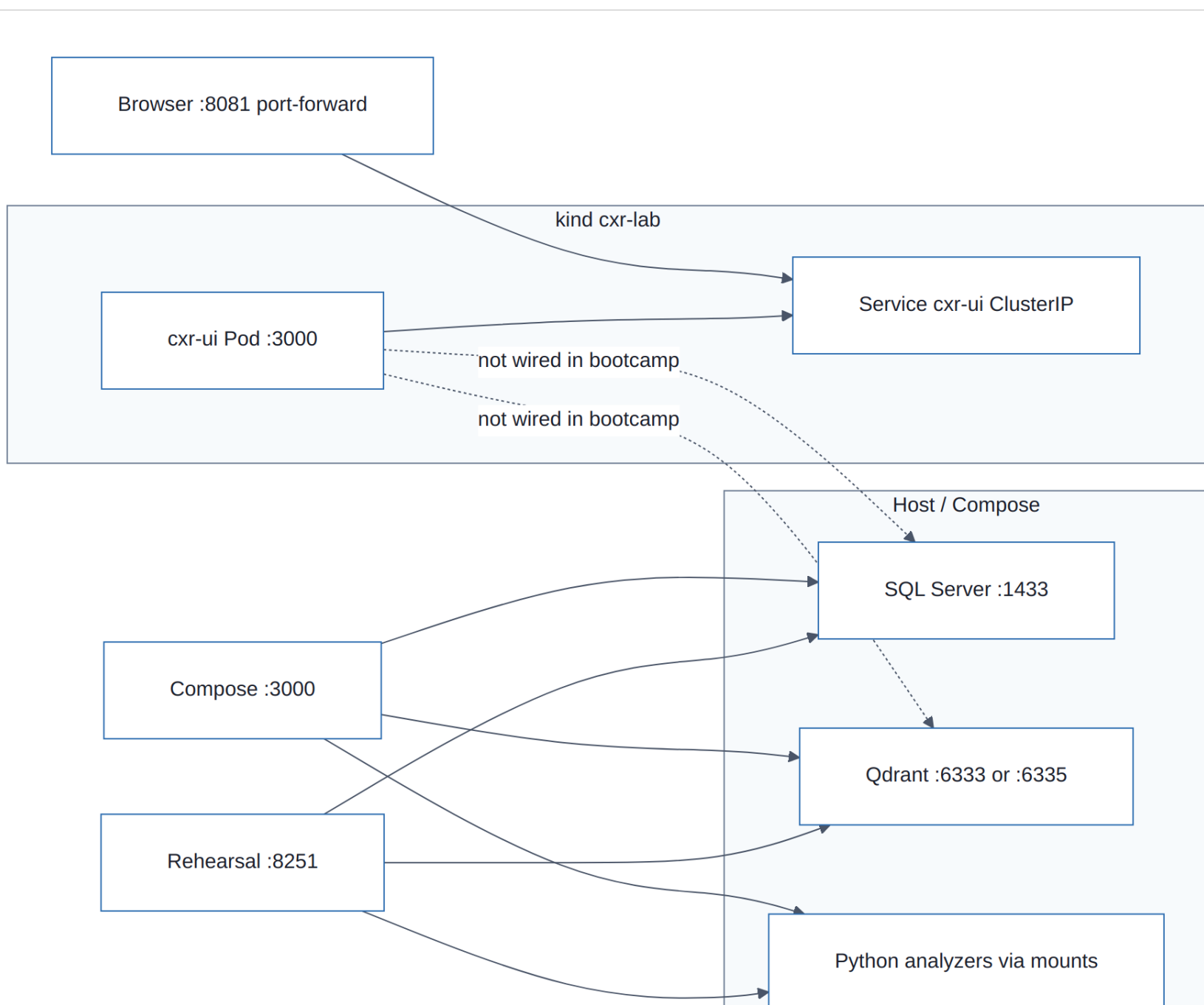


Figure 4.3: Complete CXR lab topology: rehearsal :8251, Compose :3000, K8 :8081/:8083, GitOps chart, and out-of-cluster dependencies

4.4 Dependencies (M4.8): out of cluster



SQL, Qdrant, and Python analyzers stay on the host or Compose stack in the bootcamp K8 pod. Full Claim Studio analyze uses :8251 or :3000, not the K8 UI shell alone.

Chapter 5

Step-by-step: run from zero

5.1 Prerequisites

Docker, Git, optional gh for private repo auth. Install CLIs:

```
cd /path/to/cxr-ops-lab
./scripts/00-install-tools.sh
export PATH="$PWD/bin:$PATH"
```

5.2 Option A — Full end-to-end (recommended)

```
./scripts/15-e2e-deploy.sh
# Terraform apply -> 12-k8-ensure -> 13-argo-install -> port-forwards -> 14-stack-
test
```

5.3 Option B — Manual layers

```
# SW.5 / cluster
cd terraform && terraform init && terraform apply && cd ..
# SW.1 + load
./scripts/02-build-and-load.sh
# SW.4
./scripts/05-helm-install.sh
# SW.8
export CXR_ARGO_REPO_URL=https://github.com/UdonsiKalu/cxr-ops-lab.git
./scripts/13-argo-install.sh
# Access
kubectl port-forward -n cxr-ui svc/cxr-ui 8081:3000 --address=127.0.0.1
kubectl port-forward -n argocd svc/argocd-server 8083:443 --address=127.0.0.1
```

5.4 Argo CD login

```
kubectl -n argocd get secret argocd-initial-admin-secret \
-o jsonpath='{.data.password}' | base64 -d; echo
# User: admin | URL: https://localhost:8083
```

5.5 Verify

```
./scripts/14-stack-test.sh
kubectl get application cxr-ui -n argocd
helm list -n cxr-ui
```



```
curl -s -o /dev/null -w '%{http_code}\n' http://127.0.0.1:8081/
```

Chapter 6

Helm and GitOps workflow

6.1 Helm role

Helm is the **package manager**: `helm/cxr-ui/values.yaml` sets image tag, replicas, env; templates render Deployment/Service/Namespace.

6.2 Day-to-day change loop

1. Edit `helm/cxr-ui/values.yaml` (or templates).
2. `git commit && git push` to `cxr-ops-lab`.
3. Argo Application `cxr-ui` syncs (auto-sync enabled).
4. Confirm in Argo UI: **Synced** / **Healthy**.
5. Open `http://localhost:8081` (if port-forward running).

6.3 What is not “set and forget”

- kind cluster may be stopped to save CPU.
- Image `cxr-ui:local` must be rebuilt and `kind load` after app changes.
- Port-forwards are not in Kubernetes — restart or use `cxr-k8-forward.service`.

Chapter 7

Script reference

Script	Purpose
00-install-tools.sh	Install kind, kubectl, helm, terraform, argocd into <code>bin/</code> .
01-kind-cluster.sh	Create <code>cxr-lab</code> if missing.
02-build-and-load.sh	Build <code>cxr-ui:local</code> from rehearsal UI; load into kind.
03-k8-up.sh	One-shot: cluster + build/load + Helm.
05-helm-install.sh	<code>helm upgrade -install cxr-ui</code> .
12-k8-ensure.sh	Idempotent: start kind, load image, Helm upgrade.
13-argo-install.sh	Install Argo CD; repo secret; Application from GitHub.
14-stack-test.sh	Integration test (terraform, helm, argo, HTTP probes).
15-e2e-deploy.sh	Orchestrates full stack.

Chapter 8

Adapting to another deployment

8.1 Checklist

1. Fork or clone `cxr-ops-lab`; set `CXR_UI_SRC` to your app path.
2. Replace `kind` with cloud cluster OR keep `kind` for dev.
3. Use a **container registry** (GHCR/ECR): build, push, set `values.yaml` `image.repository` and `tag`.
4. Update Argo `CXR_ARGO_REPO_URL` and private repo credentials.
5. Replace port-forward with **Ingress** or cloud LoadBalancer.
6. Wire SQL/Qdrant via in-cluster services, `ExternalName`, or `env` (see M4.8 diagram).
7. Optional: add GitHub Actions job for `helm upgrade` (push-based CD) instead of or alongside Argo.

8.2 Syllabus mapping

Syllabus	Artifact
SW.1	Dockerfile, <code>cxr-ui:local</code>
SW.2	Compose (parallel, :3000)
SW.3	<code>k8s/</code> , <code>kind</code> , :8081
SW.4	<code>helm/cxr-ui/</code>
SW.5	<code>terraform/</code>
SW.8	Argo CD + <code>k8s/argocd/</code>

Chapter 9

Troubleshooting

- **:8081 refused** — start kind node; run `12-k8-ensure.sh`; start port-forward.
- **Argo OutOfSync / private repo** — ensure `repo-cxr-ops-lab` secret + `gh auth token`.
- **Helm adopt error** — `05-helm-install.sh` adopts namespace from raw `kubectl apply`.
- **Analyze fails on :8081** — expected; use `:8251` or `:3000` Compose for full analyze.
- **ImagePullBackOff** — kind needs `kind load docker-image cxr-ui:local`.

Chapter 10

Evidence and further reading

- `evidence/SW3-k8-evidence.md`, `SW4-helm-verify-2026-05-28.md`, `SW8-e2e-stack-2026-05-28.md`
- `docs/K8-DEPLOY.md`, `docs/ADR-SW5-terraform-kind.md`
- GitHub: <https://github.com/UdonsiKalu/cxr-ops-lab>